Confluence Log in Try Confluence for free

DHAP-42 — Task Brief: Local CSV → MinIO Parquet Ingestion Pipeline

By Aufa Jul 06, 2026

At a glance — Jira: DHAP-42 (Epic) · **Status:** In Progress · **Reporter:** Nurdiansyah · **Labels:** EPIC, Interns · **Builds on:** DHAP-34 (same dataset) · **Repo:** Glynac-AI/airflow-dag-configs

- 1. Objective
- 2. Relationship to DHAP-34
- 3. Success metrics (Definition of Done)
- 4. Architecture & data flow
- 5. Repository layout
- 6. Task breakdown (Stories)
  - Story 1 — Dataset Manifest & Schema Contract
  - Story 2 — Dockerized Airflow + MinIO Environment
  - Story 3 — Airflow DAG: CSV → Parquet → MinIO
- 7. Parquet & partitioning notes
- 8. Tech stack & prerequisites
- 9. Getting started

1. Objective

Build an **Airflow-based pipeline** that ingests a CSV file (`dags/extraction/sample.csv`), **validates it against a schema contract**, transforms it, and loads it into **MinIO as partitioned Parquet**.

2. Relationship to DHAP-34

POWERED BY

Confluence Log in Try Confluence for free

destination from a relational database to an **object store in Parquet format**. It also raises the bar on validation: the **schema contract must fail the DAG on mismatch** (stricter than DHAP-34's basic validation).

3. Success metrics (Definition of Done)

- **End-to-end run:** CSV (local) → validate → transform → Parquet in MinIO (server).
- **Schema contract enforced** — the DAG **fails** if the CSV does not match.
- Parquet files are stored in MinIO and **browsable via the MinIO console**.
- Environment is **reproducible via Docker Compose**.
- Code + docs are **committed** to the repo with run steps.

4. Architecture & data flow

```
dags/extraction/sample.csv   (same dataset as DHAP-34)
       |
       v
  [ Airflow DAG, running in Docker ]
       |              |             |
   validate   ->   transform   ->  write Parquet
  (schema contract) (clean)     (partitioned)
   FAIL on mismatch                  |
                                     v
                          MinIO bucket (S3-compatible)
                          browsable in MinIO console :9001

Airflow + MinIO both run locally via docker compose.
```

5. Repository layout

```
airflow-dag-configs/
  intern-project/
    <your-name>/
      project-DHAP-42/
        <dataset_name>/
          manifest.yaml            # dataset name, source, owner, s
          schema_contract.yaml     # the contract enforced at runti
          dags/
```

Confluence Log in Try Confluence for free

```
11          .env.example
12          README.md
```

6. Task breakdown (Stories)

⚠️ The three stories below are listed in the epic. Bullet *Deliverables / Hints / Acceptance* are suggested guidance — confirm exact expectations with the reporter if in doubt.

Story 1 — Dataset Manifest & Schema Contract

Goal: define the dataset and a **schema contract** the pipeline will enforce.

Deliverables: `manifest.yaml`, `schema_contract.yaml`.

Hints: the contract should specify column names, types, nullability and (optionally) simple value rules. The DAG must **reject and fail** when the incoming CSV violates it — this is the core learning goal of Project 2.

Acceptance: a conforming CSV passes; a CSV with a missing/renamed/mistyped column fails the contract check.

Story 2 — Dockerized Airflow + MinIO Environment

Goal: one-command local stack running Airflow + MinIO.

Deliverables: `docker-compose.yaml`, `.env.example`.

Hints: add the `minio/minio` service (API `:9000`, console `:9001`); create the target bucket on startup (an init/`mc` container or a first-run task); feed MinIO credentials and the Airflow connection from `.env`.

Acceptance: `docker compose up` brings up Airflow *and* the MinIO console; you can log into the console and see the (empty) target bucket.

Story 3 — Airflow DAG: CSV → Parquet → MinIO

Goal: the DAG that reads the CSV, validates against the contract, transforms, and writes **partitioned Parquet** to MinIO.

Confluence Log in Try Confluence for free

Hints: use `pyarrow` /pandas to write Parquet; write it **partitioned** by a sensible column (e.g. a date); talk to MinIO with the `minio` client, `boto3` , or `s3fs` (S3-compatible, custom `endpoint_url`); set `catchup=False` .

Acceptance: a full run produces partitioned `.parquet` objects visible in the MinIO console; a contract-violating CSV fails the DAG before anything is written.

7. Parquet & partitioning notes

- **Format:** columnar Parquet (not CSV) — this is the lakehouse-style sink.
- **Partitioning:** lay files out as `bucket/dataset/<partition_col>=value/part-*.parquet` so the data is query-friendly and browsable.
- **Access:** MinIO is S3-compatible — point your S3 client at the MinIO `endpoint_url` with the MinIO access/secret keys.

8. Tech stack & prerequisites

- Docker & Docker Compose
- Python 3.11+, Apache Airflow 2.x
- MinIO (server + console)
- pandas, pyarrow, PyYAML, and an S3 client (minio / boto3 / s3fs)
- Git access to `Glynac-AI/airflow-dag-configs`

9. Getting started

1. Reuse the DHAP-34 dataset; place the CSV at `dags/extraction/sample.csv` .
2. Story 1: write `manifest.yaml + schema_contract.yaml` .
3. Story 2: write `docker-compose.yaml` (airflow + minio); bring it up and confirm the bucket exists.
4. Story 3: author the DAG; run until partitioned Parquet appears in MinIO.
5. Verify in the MinIO console, write the README, and open a PR.



Confluence Log in Try Confluence for free

commit under your `intern-project/` path.